

Table of Contents

Introduction.....	2
Project Objectives.....	2
System Overview.....	3
UML Diagrams.....	4
Class Responsibilities.....	5
Individual Contributions.....	8
Code Implementation.....	9
Testing & Sample Output.....	16
Challenges & Solutions.....	18
Conclusion.....	19

Introduction

This project is a **Student Information System** developed in Java using Object-Oriented Programming (OOP) principles. The system is menu-based, allowing users to interact with a clear list of options to efficiently manage student data. Users can add new students, register courses, enter grades, assign academic mentors, update information, and save/retrieve data from files.

The system consists of five main classes:

1. Login Class.
2. Student Class.
3. StudentGraddes Class (Child).
4. Mentor Class.
5. StudentMain Class.

Project Objectives

The main objectives of the Student Information System project are:

1. Efficient Student Management:

Enable users to store, update, and retrieve essential student information, including student ID, name, enrolled courses, grades, and assigned mentor if available.

2. Grades Tracking and Evaluation:

Allow entry of multiple grades per course, calculate total grades, sort grades, and determine pass/fail status for each course.

3. User Authentication:

Ensure that only authorized users can access the system through username and password verification.

4. Menu-Based Interface:

Provide a clear, menu-driven interface that allows users to easily navigate system functionalities, such as adding students, entering courses/grades, assigning mentors, displaying information, and saving data

5. Data Persistence:

Support saving student information and grades to a text file (StudentInfo.txt) with the ability to retrieve or update them safely.

6. Robustness and Error Handling:

Handle situations where no data is entered, invalid inputs are provided, or fields are missing, ensuring the system does not crash.

System Overview

The Student Information System is a Java application developed using Object-Oriented Programming (OOP) principles. The system uses a menu-based interface, guiding users through the available functionalities.

Key features of the system include:

- **Student Management:**

Users can add new students, including their basic information and academic mentor if assigned.

- **Course and Grade Management:**

Students can be enrolled in multiple courses, enter multiple grades per course, sort grades, and calculate total grades.

- **Mentor Assignment:**

Each student can have a mentor assigned, which is displayed only if a mentor exists.

- **Display and Reporting:**

The system can display all student information, including courses, grades (if entered), and mentor details in a clear, readable format.

- **File Operations:**

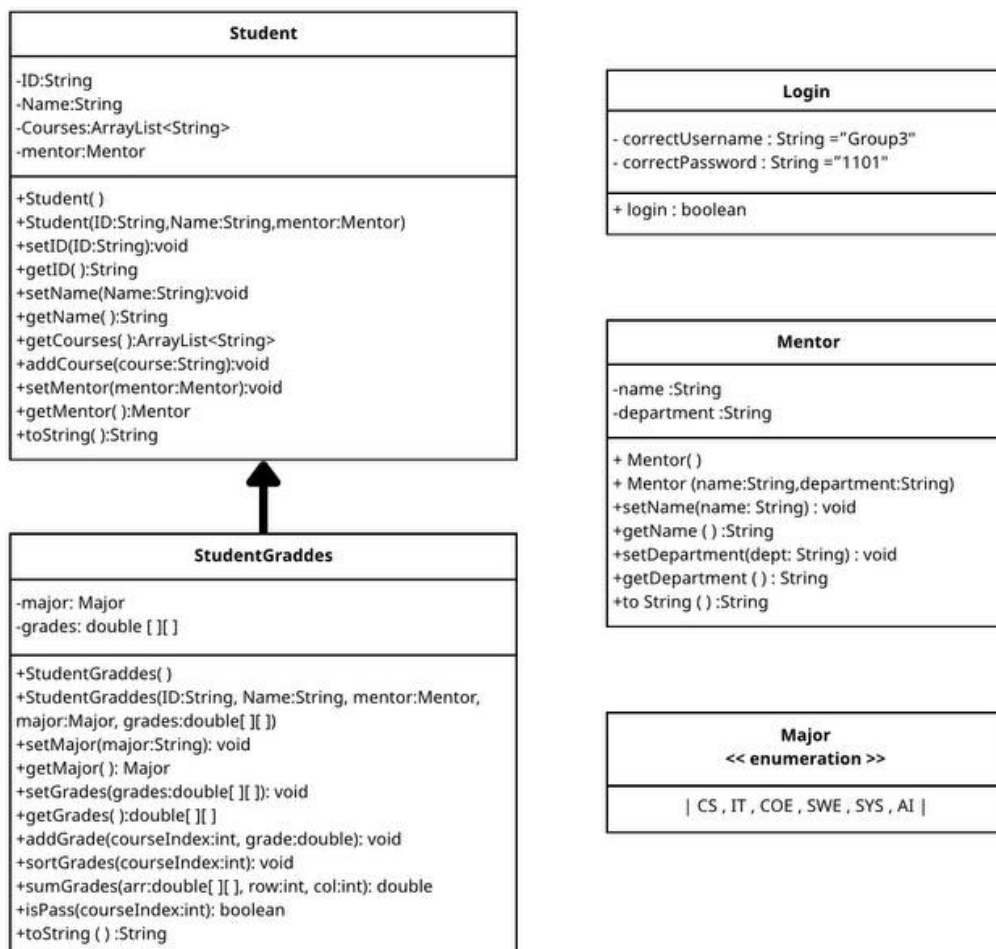
Student data can be saved to a text file (StudentInfo.txt) for record keeping, allowing data persistence across multiple program runs.

- **User Authentication:**

A simple login system ensures that only authorized users can access and modify student data.

The menu-based design makes the system user-friendly, while the separation of responsibilities across different classes (Student, StudentGraddes, Mentor, Login, StudentMain) ensures maintainability, scalability, and code clarity.

UML Diagrams



Class Responsibilities

- **Student Class :**

Description: This class serves as the foundation (Base Class) of the system, representing the general structure of the student.

Responsibilities & Functions :

Core Data Management: Responsible for storing and retrieving student identification details (ID and Name) using Setter and Getter methods.

Course Management: Utilizes an ArrayList to store the list of registered courses and provides the addCourse method to add new subjects to the course list.

Mentor Association: Maintains an object of type Mentor to link the student with their assigned academic mentor.

Design Rationale :

Designed as a Base Class to enable inheritance by the StudentGraddes class. This separates identification data from complex academic records (grades), applying the principle of Inheritance to ensure maintainability and scalability.

Utilized an ArrayList for courses instead of a fixed array to provide flexibility, allowing for the addition of an unlimited number of subjects.

Designed to include a Mentor object instead of simple text data to implement the principle of Aggregation, reflecting the "Has-A" relationship between the student and the mentor.

- **Mentor Class :**

Description: A separate class that serves as the student's academic mentor and contains data regarding faculty members responsible for advising.

Responsibilities & Functions :

- Data Encapsulation: Responsible for encapsulating mentor data, specifically the name and department.

- **Data Access:** Provides setName and setDepartment methods along with their corresponding getters to ensure safe access and modification of mentor data.
- **Display:** Provides a toString method to display mentor details in a clear, readable format when retrieving student information.

Design Rationale :

Designed as a separate class rather than embedding mentor data directly within the Student class to achieve the principle of Separation of Concerns.

Increases system flexibility, allowing for future enhancements to mentor attributes without directly affecting the Student class code.

Demonstrates the concept of Aggregation in the project, establishing that while a student has a mentor, the mentor remains an independent entity.

• **StudentMain :**

Description: The main method serves as the starting point of the program. It creates the required objects, initializes the student data, and calls the main functions to test the system.

Responsibilities & Functions:

- Initialize sample student information such as ID, name, major, mentor, and grades.
- Create objects from the classes in the project.
- Call the methods (such as adding grades, sorting, checking pass/fail).
- Display the final outputs to ensure the program works correctly

Design Rationale :

The system was designed using clear OOP principles. Each class has a specific role to keep the code organized and easy to maintain. The Student class stores the main student data, while StudentGrades handles grade calculations separately to avoid mixing responsibilities. The Main class controls the workflow, connects all classes, and manages user interaction. This structure makes the program simpler, more modular, and easier to update in the future.

- **Login class :**

Description: The Login class is responsible for handling the login process in the program. It asks the user to enter a username and password, then checks if the information is correct or not. If the credentials match the stored ones, the user is allowed to enter the program; if not, access is denied.

Responsibilities & Functions:

- This class prompts the user for login credentials.
- checks the entered username and password against the predefined correct ones,
- and decides whether to grant or deny access.
- If the credentials match, the program continues; if not, the user is prevented from entering

Design Rationale:

The Login class was designed to separate the authentication logic from the rest of the program, making the system more organized and secure. Implementing it as an independent class enhances code clarity and allows easy modification of login rules in the future.

- **StudentGraddes:**

Description: StudentGraddes represents a student with their major and course grades, allowing storage and management of multiple grades per course.

Responsibilities & Functions:

- Store and retrieve the student's major and grades.
- Add new grades and sort them for each course.
- Calculate the total of all grades recursively.
- Check if a course is passed based on total grades.
- Provide a readable string representation of the student

Design Rationale:

The class uses a 2D array for flexible grade management and handles errors safely. Recursive sum calculation and modular methods ensure clarity, reusability, and efficient grade processing.

Individual Contributions

- **Lama**

Designed and implemented the Mentor class, contributed to the Student class by implementing the course list (ArrayList) and the Aggregation, and assisted with file handling in the main method.

- **Farah**

Designed and implemented the (sumGrades) method in the StudentGraddes class. Designed the display of total grades and a file saving system that includes student information, grades and final results in the Main class.

- **Afaf**

The Login class was developed to manage user access, granting entry only upon correct username and password verification.

The StudentGraddes class, implemented as a subclass of Student, was created to store the student's major and manage their grades using a two-dimensional array. This class includes constructors for initialization and uses setter/getter methods for controlled data management.

- **Hala**

I created the Student class, including the constructor, setters, getters, and toString. I also created the enum in the StudentGrades class and assisted with the isPass and addGrades methods, and implemented the main logic, except for parts related to registration and file handling. Additionally, I refined and adjusted all code to ensure consistency and correct system functionality.

Code Implementation

- Student.java

```
package student;

import java.util.ArrayList;

public class Student {
    private String ID; // رقم هوية الطالب
    private String Name; // اسم الطالب
    private ArrayList<String> Courses; // قائمة الكورسات
    private Mentor mentor; // المشرف الأكاديمي

    // Constructor
    public Student() {
        this.ID = null;
        this.Name = null;
        this.Courses = new ArrayList<>();
        this.mentor = null;
    }

    // Constructor
    public Student(String ID, String Name, Mentor mentor) {
        this.ID = ID;
        this.Name = Name;
        this.Courses = new ArrayList<>();
        this.mentor = mentor;
    }

    // Setter Getter
    public void setID(String ID) {
        this.ID = ID;
    }

    public String getID() {
        return ID;
    }

    // Setter Getter
    public void setName(String Name) {
        this.Name = Name;
    }

    public String getName() {
        return Name;
    }

    // Getter / Setter
    public ArrayList<String> getCourses() {
        return Courses;
    }

    public void addCourse(String course) {
        this.Courses.add(course);
    }

    // Setter Getter
    public void setMentor(Mentor mentor) {
        this.mentor = mentor;
    }
}
```

```

    }

    public Mentor getMentor() {
        return mentor;
    }

    // ToString
    @Override
    public String toString() {
        return "ID: " + ID +
            "\nName: " + Name +
            "\nCourses: " + Courses +
            ((mentor != null) ? "\n" + mentor : "");
    }
}

```

• StudentGraddes.java

```

package student;

import java.util.Arrays;

// تعريف تخصص الطالب
enum Major {
    CS, IT, COE, SWE, SYS, AI
}

public class StudentGraddes extends Student {

    private Major major; // تخصص الطالب
    private double[][] grades; // مصفوفة ثنائية الأبعاد (درجات الكورسات)

    // Constructors
    public StudentGraddes() {
        super();
    }

    // Constructors
    public StudentGraddes(String ID, String name, Mentor mentor, Major major, double[][] grades) {
        super(ID, name, mentor);
        this.major = major;
        this.grades = grades;
    }

    // Setter / Getter
    public void setMajor(String major) {
        this.major = Major.valueOf(major);
    }

    public Major getMajor() {
        return major;
    }

    // Setter / Getter
    public void setGrades(double[][] grades) {
        this.grades = grades;
    }

    public double[][] getGrades() {

```

```

        return grades;
    }

    // Method إضافة درجة لـ كورس محدد
    public void addGrade(int courseIndex, double grade) {
        if (grades == null || courseIndex < 0 || courseIndex >= grades.length)
            return; // حماية من أخطاء المؤشر
        if (grades[courseIndex] == null) { // إنشاء مصفوفة جديدة إذا كانت فارغة
            grades[courseIndex] = new double[] { grade };
        } else { // توسيع المصفوفة وإضافة الدرجة الجديدة
            grades[courseIndex] = Arrays.copyOf(grades[courseIndex],
            grades[courseIndex].length + 1);
            grades[courseIndex][grades[courseIndex].length - 1] = grade;
        }
    }

    // Method ترتيب درجات الكورس تصاعديًا
    public void sortGrades(int courseIndex) {
        if (grades != null && courseIndex >= 0 && courseIndex < grades.length &&
        grades[courseIndex] != null)
            Arrays.sort(grades[courseIndex]);
    }

    // Method recursion جمع جميع الدرجات باستخدام
    public double sumGrades(double[][] arr, int row, int col) {
        if (row >= arr.length)
            return 0; // شرط التوقف

        if (arr[row] == null || col >= arr[row].length)
            return sumGrades(arr, row + 1, 0); // الانتقال للصف التالي

        return arr[row][col] + sumGrades(arr, row, col + 1);
    }

    // Method التحقق إذا الكورس ناجح أو راسب
    public boolean isPass(int courseIndex) {
        if (grades == null || courseIndex < 0 || courseIndex >= grades.length || grades[courseIndex]
        == null)
            return false;
        double sum = 0;
        for (double g : grades[courseIndex])
            sum += g;
        return sum >= 60;
    }

    // ToString
    @Override
    public String toString() {
        String result = super.toString();

        return result;
    }
}

```

• Mentor.java

```

package student;

public class Mentor {
    private String name; // اسم المشرف

```

```

private String department; // قسم المشرف

// Constructor
public Mentor(String name, String department) {
    this.name = name;
    this.department = department;
}

// Constructor
public Mentor() {
    this.name = null;
    this.department = null;
}

// Setter Getter
public void setName(String name) {
    this.name = name;
}

public String getName() {
    return name;
}

// Setter Getter
public void setDepartment(String department) {
    this.department = department;
}

public String getDepartment() {
    return department;
}

// ToString
@Override
public String toString() {
    return "Mentor: " + name +
        "\nDept: " + department;
}
}

```

• Login.java

```

package student;

import java.util.Scanner;

public class Login {

    // اسم المستخدم وكلمة السر الصحيحة
    private String correctUsername = "Group2";
    private String correctPassword = "1101";

    //Method تسجيل الدخول
    public boolean login() {
        Scanner keyboard = new Scanner(System.in);

        // ادخال اسم المستخدم
        System.out.print("Enter username: ");
        String username = keyboard.nextLine();
    }
}

```

```

// ادخال كلمة السر
System.out.print("Enter password: ");
String password = keyboard.nextLine();

// تحقق من اسم المستخدم وكلمة السر
if (username.equals(correctUsername) && password.equals(correctPassword))
{
    System.out.println("Login successful");
    return true;
} else {
    System.out.println("Incorrect username or password");
    return false;
}
}
}

```

• StudentMain.java

```

package student;

import java.io.FileWriter;
import java.io.IOException;
import java.util.Arrays;
import java.util.Scanner;

public class StudentMain {

    public static void main(String[] args) {

        Scanner keyboard = new Scanner(System.in); // لإنشاء كائن لإدخال المستخدم
        StudentGraddes s1 = new StudentGraddes(); // إنشاء كائن الطالب مع الدرجات

        // تسجيل الدخول
        Login login = new Login();
        if (!login.login()) { // التحقق من اسم المستخدم وكلمة المرور
            System.out.println("Exiting program...");
            return; // إنهاء البرنامج إذا فشل تسجيل الدخول
        }
        System.out.println("Welcome!");

        // متغير للتحكم في الحلقة الرئيسية للقائمة
        boolean exit = false;

        // القائمة الرئيسية القائمة menu-based
        while (!exit) {
            // عرض الخيارات للمستخدم
            System.out.println("\n... Student Information System Menu ...");
            System.out.println("1. Enter Student Info");
            System.out.println("2. Add Courses & Grades");
            System.out.println("3. Enter Mentor Info");
            System.out.println("4. Display Student Info");
            System.out.println("5. Save to File");
            System.out.println("6. Exit");
            System.out.print("Enter your Choice: ");

            int choice = keyboard.nextInt();
            keyboard.nextLine(); // بعد إدخال رقم الخيار buffer تنظيف الـ

            switch (choice) {
                case 1: // إدخال بيانات الطالب
                    System.out.print("Enter Your ID number : ");
                    s1.setID(keyboard.nextLine());

```

```

System.out.print("Enter Your Name: ");
s1.setName(keyboard.nextLine());

// التحقق من صحة التخصص
String major;
do {
    System.out.print("Enter Student Major (CS, IT, COE, SWE, SYS, AI): ");
    major = keyboard.nextLine().toUpperCase();
} while (!Arrays.asList("CS", "IT", "COE", "SWE", "SYS", "AI").contains(major));
s1.setMajor(major);
break;

case 2: // إضافة الكورسات والدرجات
System.out.print("Number of courses: ");
int numCourses = keyboard.nextInt();
keyboard.nextLine();

if (numCourses > 0) {
    s1.setGrades(new double[numCourses][0]); // تهيئة مصفوفة الدرجات

    for (int i = 0; i < numCourses; i++) {
        System.out.print("Enter Course " + (i + 1) + ": ");
        s1.addCourse(keyboard.nextLine());

        System.out.print("Number of grades: ");
        int numGrades = keyboard.nextInt();
        keyboard.nextLine();

        for (int j = 0; j < numGrades; j++) {
            double grade;
            do {
                System.out.print("Enter Grade " + (j + 1) + ": ");
                grade = keyboard.nextDouble();
                keyboard.nextLine();
            } while (grade < 0 || grade > 100);
            s1.addGrade(i, grade);
        }
    }
} else {
    System.out.println("No courses entered.");
}
break;

case 3: // إدخال بيانات المشرف
System.out.print("Enter Mentor Name: ");
String mName = keyboard.nextLine();

System.out.print("Enter Mentor Department: ");
String mDept = keyboard.nextLine();

s1.setMentor(new Mentor(mName, mDept));
break;

case 4: // عرض معلومات الطالب
System.out.println("\n... Student Info ...");
System.out.println(s1); // طباعة البيانات الأساسية (ID, Name, Mentor, Major)
double totalAll = 0;
if (s1.getGrades() != null) {
    totalAll = s1.sumGrades(s1.getGrades(), 0, 0);
}

```

```

    }

    System.out.println("\nGrades:");
    for (int i = 0; i < s1.getCourses().size(); i++) {
        if (s1.getGrades()[i] != null && s1.getGrades()[i].length > 0) {
            double courseTotal = 0;
            for (double g : s1.getGrades()[i])
                courseTotal += g;

            // طباعة الدرجات + المجموع الكلي للكورس + Pass/Fail
            System.out.println(s1.getCourses().get(i) + ": " +
                Arrays.toString(s1.getGrades()[i])
                + ", Total: " + courseTotal + " = " +
                (s1.isPass(i) ? "Pass" : "Fail"));
        } else {
            System.out.println(s1.getCourses().get(i) + ": No grades
                entered yet");
        }
    }

    // المجموع الكلي لجميع الدرجات
    System.out.println("\nTotal Grades (All Courses): " + totalAll);
    break;

case 5: // حفظ البيانات في ملف نصي
    try (FileWriter writer = new FileWriter("StudentInfo.txt")) {
        writer.write(s1 + "\n\n");

        if (s1.getGrades() != null) {
            for (int i = 0; i < s1.getCourses().size(); i++) {
                double total =
                    Arrays.stream(s1.getGrades()[i]).sum();
                writer.write(s1.getCourses().get(i) + ": " +
                    Arrays.toString(s1.getGrades()[i]) + " → "
                    + (s1.isPass(i) ? "Pass" : "Fail") +
                    ", Total: " + total + "\n");
            }
            writer.write("\nTotal Grades: " +
                s1.sumGrades(s1.getGrades(), 0, 0) + "\n");
        }

        System.out.println("\nFile saved successfully!");
    } catch (IOException e) {
        System.out.println("Error saving file: " + e.getMessage());
    }
    break;

case 6: // الخروج من البرنامج
    exit = true;
    System.out.println("\nExiting Program...");
    break;

default: // خيار غير صحيح
    System.out.println("Invalid choice! Please try again.");
}
}
}
}
}

```

Testing & Sample Output

```
Enter username: Group2
Enter password: 1101
Login successful
Welcome!

... Student Information System Menu ...
1. Enter Student Info
2. Add Courses & Grades
3. Enter Mentor Info
4. Display Student Info
5. Save to File
6. Exit
Enter your Choice: 1
Enter Your ID number : 123456789
Enter Your Name: Hala
Enter Student Major (CS, IT, COE, SWE, SYS, AI): Cs

... Student Information System Menu ...
1. Enter Student Info
2. Add Courses & Grades
3. Enter Mentor Info
4. Display Student Info
5. Save to File
6. Exit
Enter your Choice: 2

Number of courses: 3
Enter Course 1: Java
Number of grades: 3
Enter Grade 1: 15
Enter Grade 2: 30
Enter Grade 3: 45
Enter Course 2: Math
Number of grades: 2
Enter Grade 1: 15
Enter Grade 2: 30
Enter Course 3: English
Number of grades: 4
Enter Grade 1: 10
Enter Grade 2: 5
Enter Grade 3: 33
Enter Grade 4: 50

... Student Information System Menu ...
1. Enter Student Info
2. Add Courses & Grades
3. Enter Mentor Info
4. Display Student Info
5. Save to File
6. Exit
Enter your Choice: 3
Enter Mentor Name: Ebtehal Quqandi
Enter Mentor Department: CS
```


... Student Information System Menu ...

1. Enter Student Info
2. Add Courses & Grades
3. Enter Mentor Info
4. Display Student Info
5. Save to File
6. Exit

Enter your Choice: 4

... Student Info ...

ID: 123456789

Name: Hala

Courses: [Java, Math, English]

Mentor: Ebtehal Quqandi

Dept: CS

Grades:

Java: [15.0, 30.0, 45.0] , Total: 90.0 = Pass

Math: [15.0, 30.0] , Total: 45.0 = Fail

English: [10.0, 5.0, 33.0, 50.0] , Total: 98.0 = Pass

Total Grades (All Courses): 233.0

... Student Information System Menu ...

1. Enter Student Info
2. Add Courses & Grades
3. Enter Mentor Info
4. Display Student Info
5. Save to File
6. Exit

Enter your Choice: 5

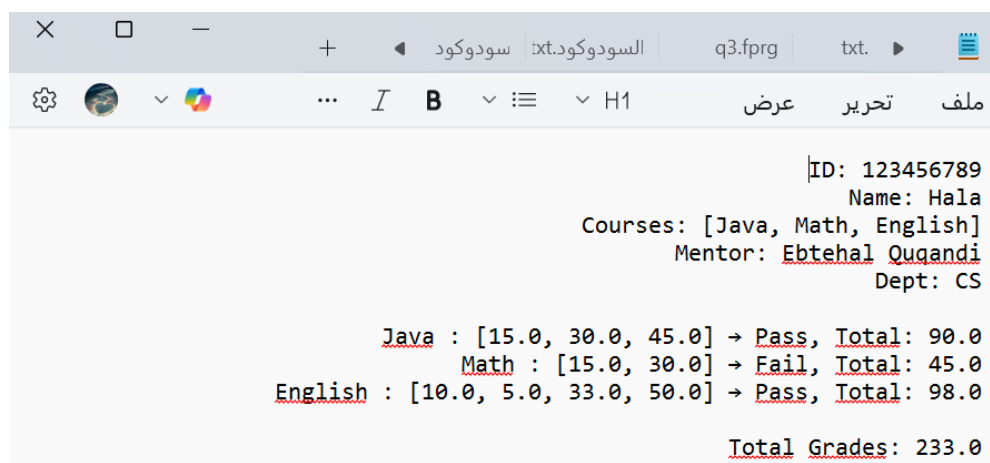
File saved successfully!

... Student Information System Menu ...

1. Enter Student Info
2. Add Courses & Grades
3. Enter Mentor Info
4. Display Student Info
5. Save to File
6. Exit

Enter your Choice: 6

Exiting Program...



The screenshot shows a text editor window with a light blue header bar. The header bar contains a close button (X), a maximize button (square), and a minimize button (dash). Below the header bar is a toolbar with icons for settings, a globe, a color palette, and a list of files. The main text area displays the following information:

```
ID: 123456789
Name: Hala
Courses: [Java, Math, English]
Mentor: Ebtehal Quqandi
Dept: CS

Java : [15.0, 30.0, 45.0] → Pass, Total: 90.0
Math : [15.0, 30.0] → Fail, Total: 45.0
English : [10.0, 5.0, 33.0, 50.0] → Pass, Total: 98.0

Total Grades: 233.0
```

Challenges & Solutions

During the development of the Student Information System, several challenges were encountered, and solutions were implemented as follows:

1. Handling Grades Input:

- Challenge: Students can have multiple grades per course, and entering them incorrectly could cause errors.
- Solution: Added validation to ensure grades are between 0 and 100, and used dynamic arrays to store multiple grades per course safely.

2. Printing Student Information When No Data Exists:

- Challenge: Displaying grades or mentor details before they were entered caused errors.
- Solution: Modified the `toString()` method to check for null values and only display data if it exists.

3. Sorting Grades:

- Challenge: Needed to sort grades for each course while handling empty or partially filled arrays.
- Solution: Used `Arrays.sort()` with null-checks to safely sort existing grades without errors.

4. File Saving and Error Handling:

- Challenge: Saving student data to a file could fail if the file path was incorrect or unavailable.
- Solution: Used try-catch blocks with `FileWriter` to handle exceptions and ensure safe file writing.

5. Menu-Based Navigation:

- Challenge: Ensuring the system is easy to use and handles invalid user inputs.

- Solution: Implemented a loop with options, input validation, and meaningful error messages for incorrect choices.

These solutions ensured the system is robust, user-friendly, and error-resistant, providing a reliable platform for managing student information.

Conclusion

This project successfully implemented a Student Information System using Java and Object-Oriented Programming principles. The Student class serves as the foundation, storing essential student information, while StudentGraddes extends it by managing specialization and grades. The Mentor class demonstrates aggregation by linking each student to an academic advisor when assigned.

The main program integrates all classes into a functional menu-based system, allowing users to enter student details, register courses, input grades, assign mentors, view information, and save data to a file.

Through this project, we enhanced our understanding of class design, inheritance, aggregation, data validation, file handling, and user input management in Java. It also highlights the importance of robust error handling and clear program structure to ensure usability and reliability.

Overall, this system provides a simple yet effective platform for managing student data, and it can be further extended in the future with features like searching, reporting, and graphical interfaces.